CodeMap AI

An interactive, agentic codebase intelligence tool

Final Assignment — AI Agent Project

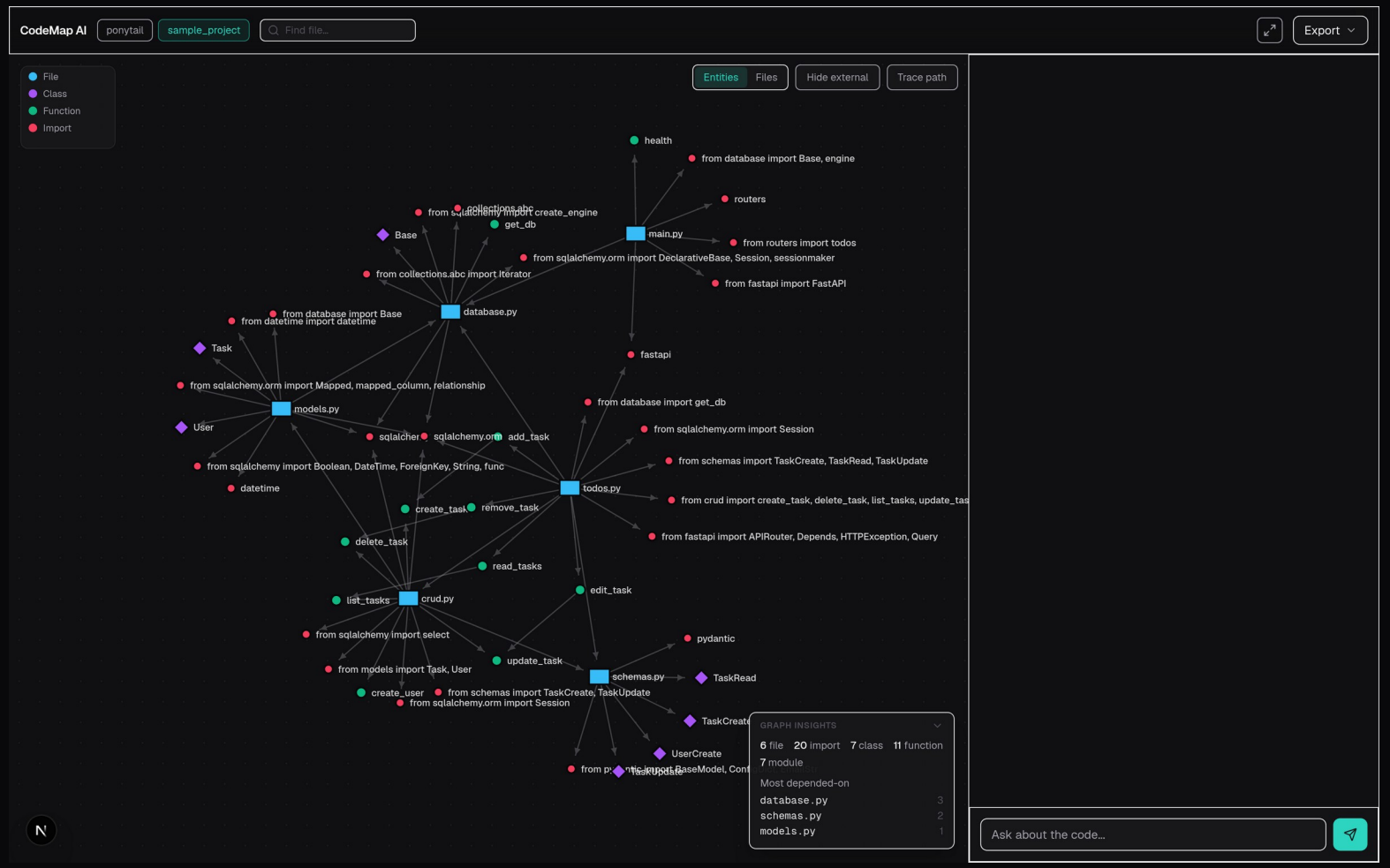
Problem Statement

- Developers spend hours onboarding to unfamiliar codebases.
- Manual exploration is slow: open file after file, trace imports, read docs.
- IDEs and grep are great for one file, poor at the big-picture architecture.
- We need a tool that parses a whole codebase, maps it, and answers questions in plain English.

Proposed Solution

- Upload any local codebase (or clone a GitHub URL).
- Parse it into an AST, build a dependency graph, and index it for semantic search.
- Explore the result in an interactive force-directed graph.
- Chat with an AI agent that reasons over the code, calls tools, and cites files and lines.

Proposed Solution — in practice



A graph-first workspace: typed dependencies, filters, and ranked hub files.

How the AI Agent Works (ReAct loop)

- Reasoning — the agent reads the question and decides what it needs.
- Thought — it plans the next step.
- Action — it calls one of seven tools: search_codebase, get_file_content, find_definition, find_references, get_dependency_path, impact_analysis, generate_diagram.
- Observation — it receives the tool result.
- Answer — it synthesises a final, cited response.
  - A live status line shows what it is doing; the answer follows.

Graph Intelligence: Blast Radius

- Edges are typed: contains, import, call. Dependency metrics ignore containment, so they are honest.
- Imports resolve for real — relative and absolute paths, packages, __init__, JS/TS extensions.
- impact_analysis(node) walks the reverse-dependency closure: everything a change would touch.
- The answer lists the affected files, and the graph lights up exactly those nodes.
- Ask: "what breaks if I change crud.py?" — the agent narrates, the graph proves it.

Architecture / Workflow

- Backend (FastAPI): tree-sitter parser → graph builder → chunker → ChromaDB → ReAct agent.
- Parser extracts classes, functions and imports from 7 languages (Python, JS/TS, Java, Go, Rust, C/C++).
- Graph builder produces file dependencies plus an entity-level call graph, with cycle detection and centrality.
- RAG indexes AST-aware chunks with line numbers and retrieves with hybrid search (vector + BM25).
- Frontend (Next.js): graph-first workspace — click a node to read its code, find any file, chat alongside.

Technologies Used

- Frontend: Next.js 16, React 19, Tailwind CSS v4, D3.js.
- Backend: FastAPI, Python 3.12.
- Parsing: tree-sitter (Python, JS/TS, Java, Go, Rust, C/C++).
- Vector store: ChromaDB (embedded) with jina-embeddings-v2-base-code.
- Retrieval: hybrid — embeddings + BM25, fused by reciprocal rank fusion.
- LLM: any OpenAI-compatible endpoint — Groq, DeepSeek, or local LM Studio.
- Resilience: automatic fallback provider when the primary is rate-limited.

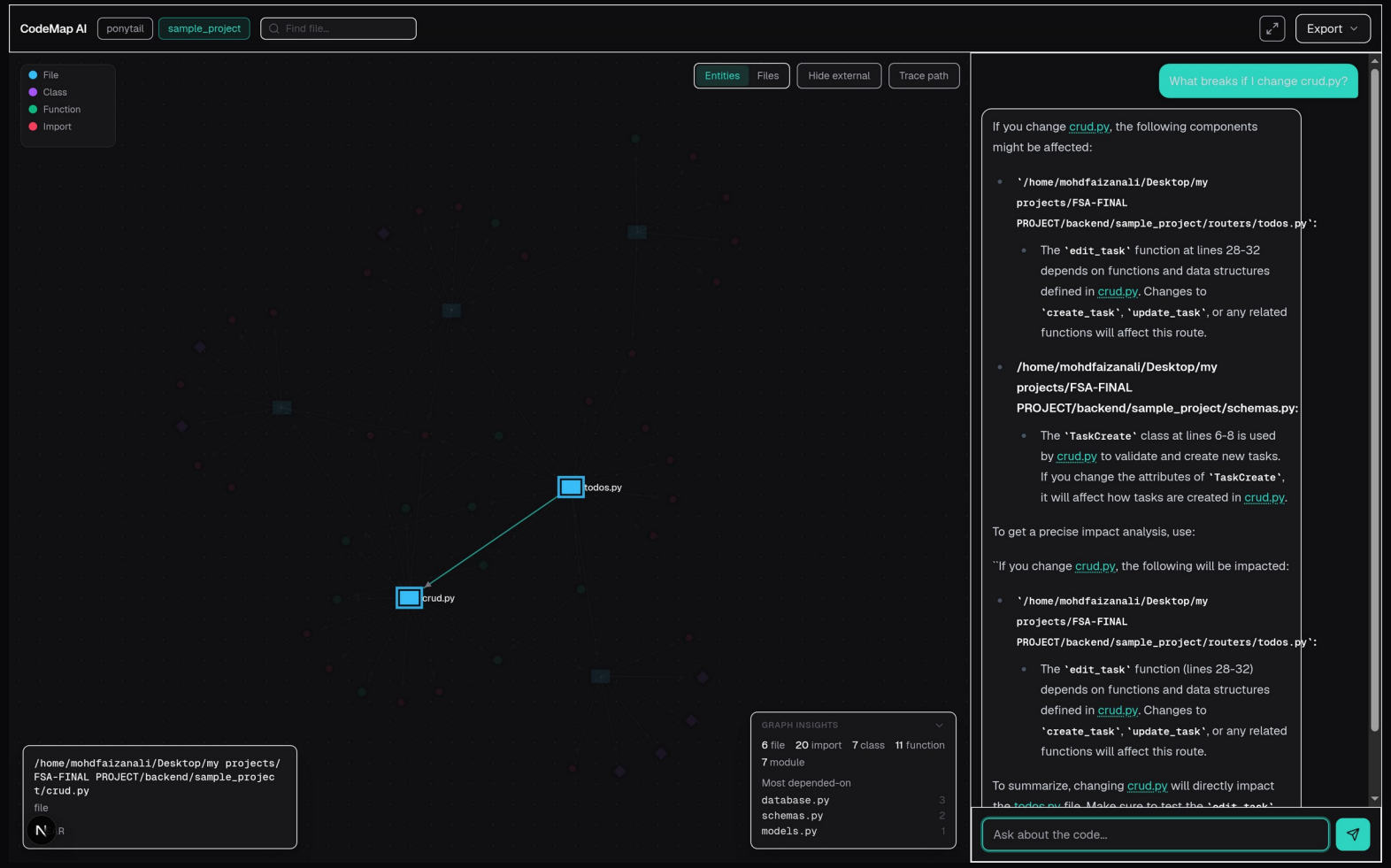
Tools / APIs Used

- Agent tools: search_codebase, get_file_content, find_definition, find_references, get_dependency_path, impact_analysis, generate_diagram.
- /api/upload and /api/clone — index a local path or a git repository.
- /api/graph and /api/graph/insights — nodes/links plus cycles and hub files.
- /api/chat/stream — the ReAct agent chat stream (SSE).
- /api/overview — LLM-generated architecture summary.
- /api/export/markdown/{id} and /api/health.

Demo

- Open http://localhost:3000 and click Load Sample.
- Explore the interactive graph: files, classes, functions, imports.
- Click a node to read its code beside the graph; search finds any file.
- Ask the chat: "What does this codebase do?"
- HEADLINE: ask "what breaks if I change crud.py?" and watch the graph light up the blast radius.
- Trace a path between two nodes: each hop shows the linking line, with Explain.
- Export the architecture as Markdown.
  - Presenting tip: verify the model with GET /api/health first.

Demo — in practice



Blast radius: the agent lists what breaks, and the graph lights up exactly those nodes.

Challenges & Learnings

- Relative imports were silently dropped: `from . import x` is a relative_import node, not dotted_name, so the query never matched.
- Containment edges polluted the graph — a file 'depended on' its own functions until edges were typed.
- Local models emit tool calls as raw text, and providers reject unparseable calls; we filter and recover in steps.
- Free tiers run out mid-session, so a fallback provider is configured and switched to automatically.
- Answers rendered badly in a narrow panel: prompts now ask for tight bullets and never pipes inside tables.

Future Improvements

- Co-change analysis: files that always change together, from git history.
- Project-level memory that survives across chat sessions.
- Incremental re-indexing: re-parse only files that changed.
- Review mode: point the agent at a git diff and get a code review.
- More export formats: PNG/SVG graph, full PDF report.

Thank you

CodeMap AI — questions welcome